

greedy & 区间 & sort

冒泡排序:

```
n = len(arr)
for i in range(n):
    swapped = False
    for j in range(0, n-i-1):
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
            swapped = True
    if not swapped:
        break
return arr
```

冒泡(双向冒泡) 2个版本排序

```
n = len(arr)
left = 0
right = n-1
while left < right:
    new_right = left
    for i in range(left, right):
        if arr[i] > arr[i+1]:
            arr[i], arr[i+1] = arr[i+1], arr[i]
            new_right = i
    right = new_right
```

example:

```
05 最大最小数
n = int(input().strip())
nums = input().strip().split()
for i in range(n-1):
    for j in range(i+1, n):
        if nums[i] + nums[j] < nums[j] + nums[i]:
            nums[i], nums[j] = nums[j], nums[i]
ans = "".join(nums)
nums.reverse()
print(ans + " " + "".join(nums))
```

贪心相同也并

区间合并(给出一堆区间,要求合并所有有交集的区间)

```
intervals.sort(key=lambda x: x[0])
merged = [intervals[0]]
for start, end in intervals[1:]:
    if start <= merged[-1][1]:
        merged[-1][1] = max(merged[-1][1], end)
    else:
        merged.append([start, end])
```

return merged

(右端点排序, 右端点尽量远, 贪心)

区间覆盖: 用给定的区间中最少的区间覆盖目标区间

```
intervals.sort(key=lambda x: x[0])
```

```
result = []
```

```
i = 0
```

```
current_end = target[0]
```

```
while i < len(intervals) and current_end < target[1]:
```

```
    max_end = current_end
```

```
    while i < len(intervals) and intervals[i][0] <= current_end:
```

```
        max_end = max(max_end, intervals[i][1])
```

```
        i += 1
```

```
    if max_end == current_end:
```

```
        return []
```

```
    result.append([current_end, max_end])
```

```
    current_end = max_end
```

```
return result if current_end >= target[1] else []
```

区间覆盖: 0.1 给定的 (给定若干段区间, 选出最少数量使其覆盖到 X 的区间)

① 必须覆盖 ② 硬币面值从小到大排序

③ 维护当前最大连续覆盖值 max_val (初始为 0)

④ 在所有满足 coin <= max_val + 1 的硬币中, 选最大的硬币, 直到 max_val == x

```
while max_reach < x:
```

```
    candidate = -1
```

```
    for coin in coins:
```

```
        if coin <= max_reach + 1:
```

```
            candidate = coin
```

```
        else:
```

```
            break
```

```
    if candidate == -1:
```

```
        return -1
```

```
    max_reach += candidate,
```

```
    count += 1
```

```
    if max_reach >= x:
```

```
        break.
```

无重叠区间: 移除最少数量的区间使剩余区间互不重叠

```
intervals.sort(key=lambda x: x[1])
```

```
count = 0
```

```
end = intervals[0][1]
```

```
for i in range(1, len(intervals)):
```

```
    if intervals[i][0] < end:
```

```
        count += 1
```

```
    else:
```

```
        end = intervals[i][1]
```

```
return count.
```

区间分组:

给出一些区间, 将其分成最少的组数, 使得组内无重叠

左端点排序

依次枚举每个区间能否放入现有的一组

若能, 更新该组; 不能, 新开一组

(现有组的右端点用 heap 存)

```
q = []
```

```
for i in range(n):
```

```
    if not q or q[0] > intervals[i][1]:
```

```
        heappush(q, intervals[i][1])
```

```
    else:
```

```
        heappop(q)
```

```
        heappush(q, intervals[i][1])
```

```
return len(q)
```

[算法经典代码]

递归

example 1: 进制转换 (十 $\rightarrow$ base)

```
def to_str(n, base):
    convert_str = "0123456789ABCDEF"
    if n < base:
        return convert_str[n]
    else:
        return to_str(n // base, base) + convert_str[n % base]
```

example 3: 波兰表达式 & 逆波兰表达式

def apply(op: str, a: float, b: float) $\rightarrow$ float:

```
if op == "+":
    return a + b
```

```
else:
    raise ValueError
```

tokens = input().strip().split()

stack = []

for tok in reversed(tokens):

```
if tok in {"+", "-", "*", "/"}:
```

```
if len(stack) < 2:
    raise ValueError
```

a = stack.pop()

b = stack.pop()

stack.append(apply(tok, a, b))

else:

stack.append(float(tok))

if len(stack) != 1:

raise ValueError

print(f"{stack[0]}")

example 4: QuickSort 递归 + "分而治之"

import random

def quicksort(arr):

if len(arr) <= 1:

return arr

else:

pivot = arr[random.randint(0, len(arr)-1)]

less = [i for i in arr[1:] if i <= pivot]

more = [i for i in arr[1:] if i > pivot]

return quicksort(less) + [pivot] + quicksort(more)

埃氏筛

import math

is_prime = [True] * (maxA + 1)

is_prime[0] = False; is_prime[1] = False

for i in range(2, int(math.sqrt(maxA)) + 1):

if is_prime[i]:

for j in range(i * i, maxA + 1, i):

is_prime[j] = False

example 2: 寻找因数分解个数

def count_factorizations(n, min_factor=2)

if n == 1:

return 1

count = 1

i = min_factor

while i * i <= n:

if n % i == 0:

count += count_factorizations(n // i, i)

i += 1

return count

for tok in tokens:

if tok in {"+", "-", "*", "/"}:

if len(stack) < 2:

raise ValueError

b = stack.pop()

a = stack.pop()

stack.append(apply(tok, a, b))

else:

stack.append(float(tok))

if len(stack) != 1:

raise ValueError

print(f"{stack[0]}")

筛法: 线性筛 & 埃氏筛

线性筛: maxA

spf = [0] * (maxA + 1)

primes = []

for i in range(2, maxA + 1):

if spf[i] == 0:

spf[i] = i

primes.append(i)

for p in primes:

v = p * i

if v > maxA:

break

spf[i] = p

if i % p == 0:

break

算法与经典代码

Heap + greedy

hp = []

i = 0

for i in range(lenh) - 1:

diff = h[i+1] - h[i]

if diff <= 0:

continue

b += diff

h = heapq.heappush(h, (-diff))

if b < 0:

b += -heapq.heappop(h)

l -= 1

if l < 0:

return i

return lenh - 1

经典用法

b: bricks.

l: ladders.

min(i, j, n - 1 - i, n - 1 - j)

决定在那一层

双指针: CF 题解 / 2016 年 10 月 sliding window

boys.sort()

girls.sort()

i, j, pairs = 0, 0, 0

while i < n and j < m:

if abs(boys[i] - girls[j]) <= 1:

pairs += 1

i += 1

j += 1

elif boys[i] < girls[j]:

i += 1

else:

j += 1

初始 right 指针不移动
且再收回 left

前缀和

eg. prefix[i] = prefix[i-1] + (1 if s[i] == s[i-1] else 0)

12: prefix[i][j] = Moscow[i-1][j-1] + prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1]

Total = prefix[x2+1][y2+1] - prefix[x2+1][y1] - prefix[x1][y2+1] + prefix[x1][y1]

求和

Sum of divisors = 0 * (n+1)

i = 1

while i <= n:

j = 2 * i

while j <= n:

sum of divisors[j] += i

j += i

i += 1

最长有效括号子串 stack

stack = [-1]
for i, char in enumerate(spell):

if char == "(":

stack.append(i)

else:

stack.pop()

if not stack:

stack.append(i)

else:

current_length = i - stack[-1]

max_length = max(current_length, max_length)

print(max_length)

DP example 1: 最长公共子序列

def longest(s1, s2):

rows, cols = len(s2), len(s1)

excel = [[0 for i in range(cols + 1)] for j in range(rows + 1)]

for i in range(1, rows + 1):

for j in range(1, cols + 1):

if s2[i-1] == s1[j-1]:

excel[i][j] = excel[i-1][j-1] + 1

else:

excel[i][j] = max(excel[i-1][j], excel[i][j-1])

return excel[rows][cols]

example 2: 最长回文子序列

dp = [0] * n

prev = -1

for i in range(n):

for j in range(i):

if nums[i] == nums[j]:
dp[i] = dp[j] + 1

max_len = max(dp)

end_idx = dp.index(max_len)

result = []

while end_idx != -1:

result.append(nums[end_idx])

end_idx = prev[end_idx]

最长回文子串:

```

if len(s) <= 1: return s
MaxLen = 1
MaxStr = s[0]
dp = [[False for _ in range(len(s))] for _ in range(len(s))]
for i in range(len(s)):
    dp[i][i] = True
    for j in range(i+1):
        if s[j] == s[i] and (i-j <= 2 or dp[j+1][i-1]):
            dp[j][i] = True
            if i-j+1 > MaxLen:
                MaxLen = i-j+1
                MaxStr = s[j:i+1]
return MaxStr

```

动态规划:

① 动态规划求最长回文子串:

```

dp = [[0] * (n+1) for _ in range(n+1)]
dp[0][0] = 1
for n in range(1, n+1):
    for k in range(1, n+1):
        dp[n][k] = dp[n-1][k+1]
        if n >= k:
            dp[n][k] += dp[n-k][k]

```

② 动态规划求最长回文子串 (O(1) 空间)

```

dp = [0] * (n+1)
dp[0] = 1
for part in range(1, n+1):
    for s in range(n, part-1, -1):
        dp[s] += dp[s-part]

```

③ 动态规划求最长回文子串

```

odd = [0] * (n+1)
odd[0] = 1
for part in range(1, n+1, 2):
    for s in range(part, n+1):
        odd[s] += odd[s-part]

```

动态规划求最长回文子串 (O(1) 空间)

```

dp = [[0] * 3 for _ in range(n+1)]
dp[0][0] = 0
for i in range(1, n+1):
    x = a[i-1]
    dp[i][0] = min(dp[i-1][0], dp[i-1][1], dp[i-1][2]) + 1
    if x in (1, 3):
        dp[i][1] = min(dp[i-1][0], dp[i-1][2])
    if x in (2, 3):
        dp[i][2] = min(dp[i-1][0], dp[i-1][1])
return min(dp[n][0], dp[n][1], dp[n][2])

```

Manacher:

```

if len(s) <= 1: return s
MaxLen = 1
MaxStr = s[0]
s = "$" + "#".join(s) + "#"
dp = [0 for _ in range(len(s))]
center = 0
right = 0
for i in range(len(s)):
    if i < right:
        dp[i] = min(right-i, dp[2*center-i])
        while i-dp[i]-1 >= 0 and i+dp[i]+1 < len(s) and s[i-dp[i]-1] == s[i+dp[i]+1]:
            dp[i] += 1
    if i+dp[i] > right:
        center = i
        right = i+dp[i]
    if dp[i] > MaxLen:
        MaxLen = dp[i]
        MaxStr = s[i-dp[i]:i+dp[i]+1],
        replace("#", "")
return MaxStr

```

动态规划求最长回文子串

```

dp = [0] * n
for i in range(n):
    for j in range(i):
        if h[i] > h[j]:
            dp[i] = max(dp[i], dp[j] + 1)
reversed_h = list(reversed(h))
reversed_dp = [0] * n
for i in range(n):
    for j in range(i):
        if reversed_h[i] > reversed_h[j]:
            reversed_dp[i] = max(reversed_dp[i], reversed_dp[j] + 1)
zip_dp = list(map(lambda x: x[0] + x[1] - 1, zip(dp, reversed_dp)))
print(max(zip_dp))

```

dijkstra 有难度的dijkstra要手动构建图

(basic版) $dist = \{ node: sys.maxsize \text{ for } node \text{ in graph} \}$

北京大学
PEKING UNIVERSITY

$dist[start] = 0$ 把权值赋给在首项!

$heap = [(0, start)]$

$previous = \{ node: None \text{ for } node \text{ in graph} \}$

while heap:

$curr_dist, curr_node = heapq.heappop(heap)$

if $curr_dist \neq dist[curr_node]$:

continue

for neighbor, delta in graph[curr_node].items():

$distance = curr_dist + delta$

if $distance < dist[neighbor]$:

$dist[neighbor] = distance$

$previous[neighbor] = curr_node$

$heapq.heappush(heap, (distance, neighbor))$

$heap = [(0, start, 0)]$ # distance, node, step

$dist = [[sys.maxsize] * (K+1) \text{ for } _ \text{ in range}(n)]$ # of nodes

while heap:

if steps < k:

for neighbor in graph:

if $new_dist < dist[neighbor][steps + 1]$:

$dist[neighbor][steps + 1] = new_dist$

$heapq.heappush(heap, (new_dist, neighbor, steps + 1))$

BFS: def BFS(graph, start):

queue = deque([start])

visited.add(start)

while queue:

vertex = queue.popleft()

for neighbor in graph[vertex]:

if neighbor not in visited:

visited.add(neighbor)

queue.append(neighbor)

return visited

if $dist[nr][nc] > dist[tr][tc] + 1$

$dist[nr][nc] = dist[tr][tc] + 1$

$path_cnt[nr][nc] = path_cnt[tr][tc]$

$prev[nr][nc] = (tr, tc)$

queue.append((nr, nc))

elif $dist[nr][nc] == dist[tr][tc] + 1$

$path_cnt[nr][nc] += path_cnt[tr][tc]$

路径恢复: $tr, tc = nr, nc$

while $(tr, tc) != (sr, sc)$:

$path.append((tr, tc))$

$tr, tc = prev[tr][tc]$

$path.append((sr, sc))$

$path.reverse()$

def solve1:

输入(sr, sc) (tr, tc) blocks

queue = deque([(sr, sc)])

moves = [(-2, +1, 1, 0), ...]

移动 到马腿 position

$dist = [[sys.maxsize] * 11 \text{ for } _ \text{ in range}(11)]$

$path_cnt = [0] * 11 \text{ for } _ \text{ in range}(11)]$

$prev = [None] * 11 \text{ for } _ \text{ in range}(11)]$

$dist[sr][sc] = 0$ $path_cnt[sr][sc] = 1$

while queue:

$r, c = queue.popleft()$

for dr, dc, br, bc in moves:

$nr, nc = r + dr, c + dc$

and (nr, nc) not in blocks

if $0 \leq nr \leq 10$ and $0 \leq nc \leq 10$ and (br, bc) not in blocks

记录路径个数:

of nodes

$path_cnt = [0] * n$

$path_cnt[start] = 1$

...

if $distance < \dots$:

更新...

$path_cnt[neighbor] = path_cnt[curr_node]$

elif $distance == \dots$:

$path_cnt[neighbor] += path_cnt[curr_node]$

如果是问到是否是target的往往有

if $curr_node == target$:

return $curr_dist$

有时候注意考虑是否是否是否是否是否

(OJ走山路)

有步数上限

k

(变式: 0-1 BFS) 0-1 BFS (输出: 1 的岛屿. 水与岛是联通所架的桥的数目) - 染色 + 0-1 BFS

PART 1: 给每个岛染色 (普通DFS)

```
def dye (matrix, x, y, n, m, idx):
    directions = [ ... ]
    queue = deque([(x, y)])
    group = [(x, y)]
    matrix[x][y] = idx
    while queue:
        cx, cy = queue.popleft()
        for dx, dy in directions:
            nx, ny = cx + dx, cy + dy
            if 0 <= nx < n and 0 <= ny < m and matrix[nx][ny] == "X":
                group.append((nx, ny))
                matrix[nx][ny] = idx
                queue.append((nx, ny))
    return group.
```

PART 3: 遍历全图找到点 (i, j) 使 $dist_1(i, j) + dist_2(i, j) + dist_3(i, j)$ 最小

输入 → 染色 → 得到 $dist_1, dist_2, dist_3$

```
ans = sys.maxsize
for i in range(n):
    for j in range(m):
        if dist0[i][j] != sys.maxsize and ... and ...:
            total = dist0[i][j] + dist1[i][j] + dist2[i][j]
            if grid[i][j] == ".":
                total += 1
            ans = min(ans, total)
```

print(ans)

(变式: 最小转弯次数 BFS → 距离不再是路径长度, 而是折成有几根线段)

```
def bfs(grid, x1, y1, x2, y2, w, h):
    dist = [[sys.maxsize] * (w+2) for _ in range(h+2)]
    queue = deque([(x1, y1)])
    dist[y1][x1] = 0
    directions = [(0, -1), (0, 1), (-1, 0), (1, 0)]
    while queue:
        cx, cy = queue.popleft()
        (current_segs = dist[cy][cx]) # 放在循环外
        for dx, dy in directions:
            nx, ny = cx + dx, cy + dy
            while 0 <= nx <= w+1 and 0 <= ny <= h+1:
                if nx == x2 and ny == y2:
                    return current_segs + 1
                if grid[ny][nx] == "X":
                    break
                if dist[ny][nx] == sys.maxsize:
                    dist[ny][nx] = current_segs + 1
                    queue.append((nx, ny))
                nx += dx
                ny += dy
    return -1
```

PART 2: 0-1 BFS

(岛权值为1, 地权值为0)

构造距离地图: 某个岛出发到某个陆地或某个岛的最大距离

```
def bfs (matrix, starts, n, m):
    dist = [[sys.maxsize] * m for _ in range(n)]
    directions = [ ... ]
    queue = deque()
    for x, y in starts:
        dist[x][y] = 0
        queue.append((x, y))
    while queue:
        cx, cy = queue.popleft()
        for dx, dy in directions:
            nx, ny = cx + dx, cy + dy
            if 0 <= nx < n and 0 <= ny < m:
                weight = 1 if matrix[nx][ny] == "." else 0
                if dist[cx][cy] + weight < dist[nx][ny]:
                    dist[nx][ny] = dist[cx][cy] + weight
                    if weight == 0:
                        queue.appendleft((nx, ny))
                    else:
                        queue.append((nx, ny))
    return dist
```

Kadane 变式: 环形数组 = 二维数组

case 1: 环形数组 → 全求, 返回最大负数
不跨界: 正常 Kadane
跨界: sum - 最小子数组

```
tot = sum(arr)
cur_max = global_max = arr[0]
cur_min = global_min = arr[0]
for num in arr[1:]:
    cur_max = max(num, cur_max + num)
    cur_min = min(num, cur_min + num)
    global_max = max(global_max, cur_max)
    global_min = min(global_min, cur_min)
if global_max < 0:
    return global_max
else:
    return max(global_max, tot - global_min)
```

Case 2: 二维数组子矩阵最大和

```
matrix / def kadane(arr):
    max_submatrix_sum = float("-inf")
    for upper in range(N):
        temp = [0] * N
        for lower in range(upper, N):
            for col in range(N):
                temp[col] += matrix[lower][col]
            current_max = kadane(temp)
            max_submatrix_sum = max(..., current_max)
    print(...)
```

[算法及经典代码]

单调栈 monotonic stack

Largest Rectangle in Histogram



mono_stack = [-1] *存储栈顶的坐标*

max_area = 0

for i in range(len(heights)):

while mono_stack[-1] != -1 and heights[i] <= heights[mono_stack[-1]]:

height = heights[mono_stack.pop()]

width = i - 1 - mono_stack[-1]

max_area = max(max_area, height * width)

mono_stack.append(i)

while mono_stack[-1] != -1:

height = heights[mono_stack.pop()]

width = len(heights) - mono_stack[-1] - 1

max_area = max(max_area, height * width)

北京大学
PEKING UNIVERSITY

变式: 01 矩阵中的最大全为 1 的正方形



按列遍历作为直方图中最大矩形的面积问题

n = len(matrix[0])

height = [0] * n

res = 0

for row in matrix:

for i in range(n):

height[i] = height[i] + 1

if row[i] == "1" else 0

stack = []

for i in range(n+1):

h = height[i] if i < n else 0

while stack and h <= height[stack[-1]]:

top = stack.pop()

width = i if not stack

else i - 1 - stack[-1]

res = max(res, width * height[top])



stack = []

water = 0

for i, h in enumerate(heights):

while stack and h <= heights[stack[-1]]:

top = stack.pop()

if not stack: *处理左右边界情况*
break

left = stack[-1]

width = i - left - 1

bounded_height = min(h, heights[left]) - heights[top]

water += width * bounded_height

stack.append(i)

2. 双指针法:

left = 0

right = len(height) - 1

left_max = height[left]

right_max = height[right]

water = 0

while left < right:

if left_max < right_max:

left += 1

left_max = max(left_max, height[left])

water += left_max - height[left]

else:

right += 1

right_max = max(right_max, height[right])

water += right_max - height[right]

单调栈解决滑动窗口内最大数问题

dq = deque() *存放下标, 保持对应值递减*

res = []

for right, x in enumerate(nums):

while dq and nums[dq[-1]] <= x:

dq.pop()

dq.append(right)

if dq[0] <= right - k:

dq.popleft()

if right >= k - 1:

res.append(nums[dq[0]])

窗口有数, 保持递减

移除滑出窗口的左端元素

窗口形成 (>= k)

记录最大值

并查集 (按秩合并 + 路径压缩)

class UnionFind:

```
def __init__(self, n):
    self.parent = [i for i in range(n)]
    self.rank = [0] * n
```

若不压缩 while self.parent[x] != x:
 $x = \text{self.parent}[x]$
 return x
 递归路径压缩

```
def find(self, x):
    if self.parent[x] != x:
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]
```

```
def union(self, x, y):
    root_x = self.find(x)
    root_y = self.find(y)
    if root_x == root_y:
        return
    if self.rank[root_x] < self.rank[root_y]:
        self.parent[root_x] = root_y
    elif self.rank[root_x] > self.rank[root_y]:
        self.parent[root_y] = root_x
    else:
        self.parent[root_y] = root_x
        self.rank[root_x] += 1
```

按秩合并

升级: 是题目要考虑不同类别之间关系 (OJ 的题目)

建立 $\text{diff}[i] = (\text{type}[i] - \text{type}[\text{parent}[i]]) \bmod 3$

```
parent = list(range(N+1))
```

```
diff = [0] * (n+1)
```

```
def find(x):
```

```
    if parent[x] != x:
```

```
        orig_parent = parent[x]
```

```
        parent[x] = find(parent[x])
```

```
        diff[x] = (diff[x] + diff[orig_parent]) % 3
```

```
    return parent[x]
```

```
def union(x, y, r): # 类型 (type[x] - type[y]) mod 3 == r
```

```
    rx = find(x)
```

```
    ry = find(y)
```

```
    if rx == ry:
```

```
        return (diff[rx] - diff[ry]) % 3 == r
```

```
    parent[rx] = ry
```

```
    diff[rx] = (r - diff[rx] + diff[ry]) % 3
```

```
    return True
```

变式: 求 kadane

```
curr_max = arr[0]; curr_min = arr[0]; res = curr_max
```

```
for i in range(1, len(arr)):
```

```
    curr = arr[i]
```

```
    temp_max = max(curr, curr_max + curr, curr_min + curr)
```

```
    curr_min = min(curr, curr_max + curr, curr_min + curr)
```

```
    curr_max = temp_max
```

```
    res = max(res, curr_max)
```

Kadane → "最大子数组和"

```
def kadane(arr):
```

```
    curr_sum = max_sum = arr[0]
```

```
    for num in arr[1:]:
```

```
        curr_sum = max(num, curr_sum + num)
```

```
        max_sum = max(max_sum, curr_sum)
```

```
    return max_sum
```

变式: 可删除 1 个数的 kadane

$f = \text{curr}$ 删掉最大
 $g = \text{curr}$ 删掉最大
 $\max(f, g)$ 为全局最优

```
f = goods[0]; g = float("-inf"); ans = goods[0]
```

```
for x in goods[1:]:
```

```
    g = max(g + x, f) → 删掉当前元素或延续已删掉的
```

```
    f = max(f + x, x)
```

```
    ans = max(f, g, ans)
```

```
return ans
```

DFS & backtrack

basic 模板:

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
    return visited
```

example 1 0-1 矩阵中有多少个孤立的 "1" ?
(0 代表 200 Number of Islands)

```
m, n = len(grid), len(grid[0])
def dfs(pos):
    x, y = pos
    current = grid[x][y]
    if current == "1" or current == "0":
        return False
    grid[x][y] = "1"  # 标记 visited 防止重复
    for x_offset, y_offset in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        nx, ny = x + x_offset, y + y_offset
        if 0 <= nx < m and 0 <= ny < n:
            dfs((nx, ny))
    return True
```

res = 0

```
for i in range(m):
    for j in range(n):
        if grid[i][j] == "1":
            if dfs((i, j)):
                res += 1
```

return res

example 3: n 皇后

result = []

def solve_n_queens(n):

def backtrack(row, cols, diag1, diag2, path):

if row == n:

result.append(path)

return

for col in range(n):

if col in cols or (row - col) in diag1 or (row + col) in diag2:

continue

cols.add(col)

diag1.add(row - col); diag2.add(row + col)

path += str(col + 1)

backtrack(row + 1, cols, diag1, diag2, path)

cols.remove(col)

diag1.remove(row - col); diag2.remove(row + col)

path = path[:-1]

北京大学
PEKING UNIVERSITY

def backtrack(path, 选择列表):

if 满足条件:

res.append(path)

return

for 选择 in 选择列表:

if 选择 not in 路径: (并剪枝条件)

做选择

path.append(选择)

进入下一层决策树

backtrack(path, 选择列表)

撤销选择

path.pop()

example 1 全排列问题 (各个字母都不同)

s = input().strip; n = len(s)

used = [False] * n; path = []

def dfs():

if len(path) == n:

print(" ".join(path))

return

for i in range(n):

if not used[i]:

used[i] = True

path.append(s[i])

dfs()

path.pop()

used[i] = False

dfs()

example 2 给定的 array 并不是所有元素互不相同如何排列

used = [False] * len(nums)

def backtrack():

if len(path) == len(nums):

res.append(path[:])

return

for i in range(len(nums)):

if used[i]:

continue

if i > 0 and nums[i] == nums[i-1] and not used[i-1]:

continue

used[i] = True

path.append(nums[i])

backtrack()

path.pop()

used[i] = False

backtrack()

return res

△ 解法: 矩阵中的一个路径问题 (有障碍物/无环/求最短路)

DFS + DP (记忆化搜索) VS BFS + 拓扑排序 (Kahn算法, 适用于 DAG) example: 滑雪 (最长下降路径)

记忆化搜索 (DFS + DP)

```

r, c = map(int, input().strip().split())
field = []
for row in range(r+2):
    if row == 0 or row == r+1: # 只是边界保护
        field.append(['0000'] * (c+2))
    else:
        line = list(map(int, input().strip().split()))
        field.append(['0000'] + line + ['0000'])
dp = [[0] * (c+2) for _ in range(r+2)]
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

def dfs(x, y):
    if dp[x][y]:
        return dp[x][y]
    for dx, dy in directions:
        nx, ny = x + dx, y + dy
        if field[nx][ny] < field[x][y]:
            dp[x][y] = max(dp[x][y], dfs(nx, ny) + 1)
    if dp[x][y] == 0: # 局部最小值
        dp[x][y] = 1
    return dp[x][y]
    
```

Kahn算法 (BFS + 拓扑排序) r, c 行/列

地图: height (= 值 list)

```

def neighbor(i, j):
    lst = []
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dx, dy in directions:
        if 0 <= i + dx < r and 0 <= j + dy < c:
            lst.append((i + dx, j + dy))
    return lst

queue = deque()
deg = defaultdict(int) # deg: 入度
for i in range(0, r):
    for j in range(0, c):
        for (cx, cy) in neighbor(i, j):
            deg[(cx, cy)] += 1
            queue.append((cx, cy))

step = 0
while queue:
    for _ in range(len(queue)):
        x, y = queue.popleft()
        for nx, ny in neighbor(x, y):
            if height[nx][ny] > height[x][y]:
                deg[(nx, ny)] -= 1
                if deg[(nx, ny)] == 0:
                    queue.append((nx, ny))
    step += 1
    print(step)
    if len(queue) == 0:
        break
    
```

Kahn算法也可以用于
检查是否有环

节点数

Binary Search

example: 01 月度开销

n 个数 → 相邻两数合并, 要合并成 m 个最大值最小的集合。

```

def check(x):
    num, cut = 1, 0
    for i in range(n):
        if cut + L[i] > x:
            num += 1
            cut = L[i]
        else:
            cut += L[i]
    if num > m:
        return False
    else:
        return True
    
```

maxmax = sum(L)

minmax = max(L)

while minmax < maxmax:

mid = (minmax + maxmax) // 2

if check(mid):

maxmax = mid

else:

minmax = mid + 1

print(maxmax)

Next Permutation

(字典序)

(给出数组 nums, 要求按后 k 个排列)

```

def next_permutation(nums):
    idx = len(nums) - 2
    while idx >= 0 and nums[idx] >= nums[idx + 1]:
        idx -= 1
    if idx == -1:
        nums.reverse()
        return
    j = len(nums) - 1
    while nums[j] <= nums[idx]:
        j -= 1
    nums[idx], nums[j] = nums[j], nums[idx]
    nums[idx + 1:] = reversed(nums[idx + 1:])
    return
    
```

for _ in range(k)

python 内置 next_permutation (num)

(可以不用自己实现)

import itertools:

n, k = map(int, input().split())

arr = tuple(map(int, input().split()))

perms = list(itertools.permutations(range(1, n+1)))

idx = perms.index(arr)

result = perms[(idx + k) % len(perms)]

print(*result)